FABRID-IDS: Federated Alert-Budget Reallocation for Heterogeneous IoT Intrusion Detection

Salaheddine Naslouby [1*], Mohamed Lachgar[2] and Hamid Hrimech[1]

[1]Hassan First University, National School of Applied Sciences (ENSA), LAMSAD Laboratory, Berrechid, Morocco.
[2]Cadi Ayyad University, Higher Normal School (ENS), L2IS Laboratory, Marrakech, Morocco.

*Corresponding author(s). E-mail(s): naslouby.salahe@gmail.com;
Contributing authors: m.lachgar@uca.ac.ma; hamid.hrimech@uhp.ac.ma;

Abstract

High pooled rank performance can occur with anomaly scores that give bad per-client performance once thresholds are set. In this paper we look at the distribution of a federation nominal FPR budget after the detector has been trained. Nominal client targets are optimized together and then turned into thresholds using a disjoint set of benign samples. We test our approach on 9 N-BaIoT clients with 10 detector seeds and 5 budgets. A second setup with 19 CIC IoT-DIAD clients uses a different set of features and ranker. Our two unrestricted allocation methods (Macro and Tail) achieved a heldout federation FPR of 0.00170 when B = 0.001. This is 1.696 times the budgeted FPR. Calibrating our methods more conservatively allowed us to lower the FPR to 0.00036 while sacrificing some macro recall (0.3778→ 0.2445). Equal-FPR gave better macro recall at some budgets but also over shot its budgeted FPR at some budgets. When looking at rank we can see very high values for pooled AUROC and AUPRC of 0.9995 and 0.9999 respectively. However across all FABRID policies and budgets there was a client that had 0 recall for attack on N-BaIoT. This highlights a weakness in trying to distribute a nominal FPR across clients. It will not allow us to guarantee the federation FPR or detect attacks on every client.

Keywords: federated intrusion detection, Internet of Things, alert-budget reallocation, false-positive-rate constraint, threshold calibration

1 Introduction

After we train our detector we get an anomaly score. Once the score exceeds the threshold we will send an alert. Deciding how to set this threshold can be hard when the clients that take part in federated learning have different benign score distributions. Or there could be variations in how attacks are scored. Federated learning for IoT Intrusion detection systems has been researched for both global and client specific thresholds [1–6]. What about choosing which clients to allocate a higher or lower nominal FPR if a budget is given to the federation?

This paper focuses on setting nominal client FPR's after the anomaly scores have been learned. We will use validation attacks and benign frontier scores to choose these client FPR's (α_k). $\sum_k w_k \alpha_k \leq B$ will keep our allocation of FPRs below our desired budget B. Clients will then use a different set of benign samples to find their threshold for sending alerts. Our held out test data can evaluate both our choice of nominal FPR as well as our ability to set these thresholds from a limited set of samples. Budget B will only effect how high we can set our nominal FPR allocations not the actual number of false positives that will occur.

We want to see how well we can set different nominal FPR's whose weighted sum is at most our given federation budget. How does this nominal FPR compare to what we can achieve when finding client thresholds using our calibration samples. Lastly we want to know how each of our allocation methods can detect attacks on average and for the worst client. All of our N-BaIoT allocation methods will be tested on the same anomaly scores and detector seeds.

2 Related work and scope

It is helpful to differentiate between how a detector is trained and what alert policy is applied when placing FABRID-IDS in context. Much of the literature on federated IoT IDS has focused on learning architectures and aggregation mechanisms [1–3, 7]. There are also works that look at anomaly thresholds and detecting at the edge [8, 9]. Laridi et al. [4] pool summary statistics from the client to determine an overall anomaly threshold. Asiri et al. [5] obtain local quantiles for the error of reconstructing benign traffic and pool those thresholds. [6] obtains a local threshold based on a holdout of benign data at a desired local FPR. These are all related thresholding approaches or offer useful context but none look at allocating the federation nominal FPR.

Restricting error rates can also be seen in Neyman-Pearson type classification problems [10]. Conformal methods use calibration ranks to provide statements about finite sample performance given some assumption about sampling [11]. There is research looking at sharing predictors and aggregating calibration scores in a federated setup but address a slightly different problem [12]. FABRID-IDS uses these calibration results as a black box and decides on the nominal targets before the thresholds are found. We focus on allocating the nominal FPR across the federation and show what that allocation produces.

There are other works on fairness at training time and agnostic federated learning which try to modify the loss function or effect of a client [7]. FABRID-IDS does not do this. We can alter the value of α_k after training is complete and this will effect our

decision threshold for client k but will not change its ranking of benign or attack traffic. We will see in the results section that this is important as we can compare different threshold policies with the same scorer. If our scorer does not perform well, all of our policies will suffer. Changing the nominal rates does not allow us to detect an attack that our scorer would always put under the threshold.

We keep three ideas separate when discussing thresholding. First you can have a global absolute threshold and compare everything against that value. Second you can allow local thresholds but specify a nominal FPR across all clients (Equal-FPR). Finally like Equal-FPR we calibrate thresholds locally but can allocate the FPR based on our weighted constraint.

3 FABRID-IDS

3.1 Protocol and utility

We denote the frozen scoring function for client k as $s_k(\cdot)$. Our data is split into training 50%, find frontier 20%, final calibration 10% and test 20% using the file order we were given for the benign data. Each attack subtype will reserve its first 20% as validation with the rest as test. When allocating the nominal rates we have access to the frontier points and attack validation points. After allocation we use the final calibration points to determine a threshold and the test points are unused. These percentages are how we were able to run our benchmark protocol and are not known to reflect real world splits of time.

Given a sample of size n of benign data in our calibration set. Let $x_{(1)} \leq ... \leq x_{(n)}$ be its order statistics. Given a nominal rate alpha we will map it to a threshold by equation 1 [11].

$$\tau(\alpha) = x_{(r)}, \qquad r = \lceil (n+1)(1-\alpha) \rceil \,, \tag{1}$$

When alpha ≤ 0 we set tau = infinity or when n = 0 or r is greater then n. An alert will occur if the score is greater than tau. We apply this to our frontier points and nominal rate to find a temporary threshold. We then compute $u_k(\alpha)$ which is the mean recall of each valid attack subtype for client k. We require attack labels to create this function. Our search space for alpha is a grid of 207 values from 0 to .05 and includes our desired budgets. Subtypes must have 50 validation points to be considered and a client needs 200 validation points across at least 2 subtypes.

When finding model parameters we use the training split of benign points for each client. Once an allocation has been chosen we throw out the thresholds from our frontier and use Eq. 1 on our final calibration set. Test points are never used. While we are not using our frontier points to find a final threshold it does not mean that they have the same distribution as our calibration and test points.

Our nominal rates will be closer together than what we can get from our sample of benign points. If we choose multiple α_k's that lead to the same value for x_r we are technically allocating them different rates. But we will get the same result on our test set as our scoring function is static. So we should interpret any change in nominal rates as doing something different only if it leads to a change in our threshold.

To find our recall for attacks we first compute it for each valid subtype and take the mean on the client level. Our endpoint of federation macro recall is the mean of all our valid clients. And our last endpoint is the minimum value of our client level means. We will report our FPR on the hold out set of benign points as well.

3.2 Allocation and calibration

We will find our nominal rates from our given grid of candidates. Our choices must satisfy that the sum of the weights times each nominal rate is less than or equal to our budget.

$$\sum_{k=1}^{K} w_k \alpha_k \leq B. \tag{2}$$

Where $w_k = 1/\mathrm{K}$ for N-BaIoT so our budget is an average across our clients. We solve this as a mixed integer problem by having the allocator pick one alpha value for each client. We use the HiGHS library to solve for our Macro and Tail objective functions. Macro tries to maximize the average of u_k. Tail will maximize the mean of the bottom q = ceil(0.25K) values of u_k [13]. As we have 9 devices q=3. It is similar to empirical CVaR but will have some numerical differences as it is taking the average of the bottom three values instead of applying a fractional weight to the third value [13]. Each point in our frontier will have some minimal FPR they can generate. So if n_f is our number of frontier points for the benign class we will discard all positive candidate alpha values that are less than $2/(n_f + 1)$. Since they can't generate a positive FPR when calibrating.

Do not confuse this definition of equal with using all your benign points and finding a global FPR. Eq. 2 states that you can not exceed your budget. You can choose to underutilize your budget and assign a client 0 if the utility gained is not worth the trade off of increasing their nominal rate.

For each problem we will have a list of criteria to optimize for. For Macro we first optimize for our mean estimated utility. Then we minimize the nominal amount we spent subject to the solvers notion of optimality. For Tail we optimize for the mean of our bottom three utilities. Then optimize for our mean estimated utility and finally minimize our nominal spending. We will use the variance of our estimated utilities and then some form of deterministic tie break to find our solution. Note we are only using our frontier points and validation attacks to find these allocations. Equal-FPR simply gives each client a value of B. All our unrestricted policies will share the same benign points and Eq. 1. To find our final thresholds for the conservative Macro and Tail we will use Eq. 3.

One should note that by optimizing for a higher mean of the bottom three utilities does not mean we will protect our worst client better. Say the bottom three devices each have an estimated recall of 0.2. If we increased two of those devices by 0.1 and left one alone our objective would be improved but that one device could still fail to detect any attacks in our evaluation.

When looking for a more conservative way to calibrate we look at [10] and find the rank we need to achieve a 95% confidence in our population FPR.

$$r_{.95} = \min\{r : \Pr[\text{Binomial}(n, 1-\alpha) \leq r-1] \geq 0.95\}. \tag{3}$$

For each client we can search from rank 1 to n until we find a value that meets our criteria. If we never reach our criteria or have n = 0 or alpha = 0 we will again have no alerts by setting our threshold to infinity. Otherwise an alert occurs if s > tau. If we assume i.i.d sampling for our calibration and future score values we can make a one-sided 95% confidence statement about the population FPR for each client [10]. It should be noted this is only about the sampling distribution and not that we know our test partition is a sample from that distribution. We evaluate this as a secondary result to show that if you were to allocate in a federated manner and use a more conservative way to find thresholds it is possible to exceed your budget.

As our rank will likely be higher than Eq. 1 we may decrease our false alarms but may increase our false negatives. We are not using our test labels to make decisions on a threshold. So if we have different behavior in our benign scores between our calibration set and test set we will not be able to correct for it.

If you find that one of your clients exceeds there nominal FPR on a hold out set of data. That does not mean they violated our constraints when allocating. Or if one of your clients stays under there allocated FPR on a partition that does not mean it will stay under in the future. Which is why we track our allocated nominal FPR, rank on our final calibration set and the FPR we observe on our hold out data. All our unrestricted policies will just vary in the nominal rates they assign to each client.

Our minimal nonzero FPR on our calibration set of n points without any ties is 1/n. So any value below 2/(n+1) will give us an r that is out of range or find the max of our points. So any rate below this threshold will give us 0 alerts on our calibration set. This means that our grid of candidates will have some nominal rates that have no effect on our FPR. For our ecobee device we have 1,311 points so any rate below 0.001524 will produce no alerts during calibration. Again this may still give us a positive FPR on our test set but no alerts during calibration.

3.3 Frozen scorer

For each device on N-BaIoT we create a hybrid scoring function using the reconstructed error from our AE and `HH_jit_L0.1_mean`(f_{75}). We take the percentile of these two values using a fixed reference from the frontier and take the max. Our feature and scorer were chosen from development splits using predeclared development-only ablations and not modified when experimenting with different threshold policies. Our AE had a hidden layer size of [64, 16] and we trained for 10 rounds of federated learning. Taking 3 local epochs in each round. We used Adam with a learning rate of 0.001 and a batch size of 64 and MSE loss. We weighted our clients by the amount of rows they had for training. And used our training split to normalize the clients. We used 10 seeds for our detectors and will always use the same scores for each policy. Meaning we can not affect AUROC and AUPRC by modifying our threshold.

4 Experimental methodology

We focus on using the N-BaIoT dataset to evaluate FABRID-IDS. Each of its 9 devices will be used as our clients and we will test with budgets of B=0.001, 0.0025, 0.005, 0.010 and 0.020. Our data will contain benign and BASHLITE and Mirai attacks [14]. The amount of data varies greatly per client with ecobee having 13,113 points of benign and philips baby monitor having 175,240. So when splitting the data to find our final threshold each device will have different amounts of precision. Macro recall is computed on the subtypes of each client and then we find the mean of that value. Our worst client is just the min of these values.

For each seed, all policies use the same detector and scoring data. So when comparing our results from FABRID-IDS Macro and Equal-FPR we are comparing within that seed. We will have 10 total values from our 10 seeds and can find the mean across those seeds. This mean is from the random initialization and training process not a mean over newly found clients or periods of time.

We use the same weights from Eq. 2 to compute our hold out federation FPR. Which means that while our allocation respects the nominal budget, the measured FPR may not match. Due to how our data is split and the windows used to create features. Training with different seeds will not solve this problem.

On N-BaIoT we decided to compare Macro and Equal-FPR on federation recall and Tail and Equal-FPR on min client recall. We find our outcome on each seed and then subtract Equal-FPR. Allowing us to keep the pairings of the values. Our data points are not replicated by having different amounts of data or attack types to detect on.

Using the paired sign flip test we can enumerate through all 2^{10} ways of assigning our 10 values to a negative or positive. To create our confidence interval we will sample with replacement from those values 50,000 times, compute the mean for each sample, and find the 2.5th and 97.5th percentiles of those means. We then p value adjust our budgets with Holms. This gives us a representation of how our detector changes with 10 different training scenarios. Not that we are able to generalize this to new clients.

The CIC IoT-DIAD dataset is created by Rabbani et al. [15] while working on device identification and anomaly detection. We use this as a second example of our protocol. It has 19 devices that we can use as clients and 57 features. Similar to N-BaIoT we will train a detector on this dataset. Our benign split and % of attacks used for validation will be the same. We do not carry over any information or detector from N-BaIoT. As everything from data to client is different we can not say what caused any discrepancy in FPR.

5 Results

5.1 Nominal and realized operating points

When B is set to 0.001 the unrestricted forms of Macro and Tail give the same realized federation FPR of 0.00170 (1.696B). If we use the more conservative version of these two the FPR drops to 0.00036 (0.358B) but the macro recall goes down to 0.2445 from 0.3778. The nominal FPR is kept at or below the set budget for all four methods. However the realized FPR is what we get when we look at held out benign samples.

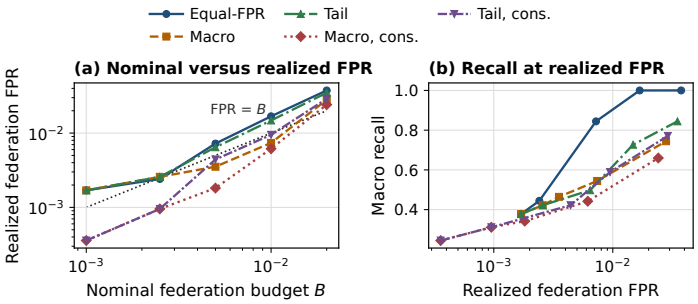


Fig. 1 N-BaIoT ten-seed averages. Left shows realized FPR vs the nominal and right shows the macro recall vs realized FPR. Diagonal line shows equal rates.

Table 1 N-BaIoT ten-seed averages. Cell shows the macro recall and the realized federation FPR / Nominal budget. FABRID family methods have a min client recall of zero.

B	Equal-FPR	Macro	Tail	Macro, cons.	Tail, cons.
0.0010	0.3778; 1.70	0.3778; 1.70	0.3778; 1.70	0.2445; 0.36	0.2445; 0.36
0.0025	0.4445; 0.97	0.4222; 1.03	0.4222; 1.03	0.3111; 0.38	0.3111; 0.38
0.0050	0.8444; 1.45	0.4644; 0.71	0.4977; 1.29	0.3422; 0.36	0.4222; 0.88
0.0100	0.9999; 1.68	0.5444; 0.74	0.7266; 1.48	0.4422; 0.62	0.5889; 0.95
0.0200	1.0000; 1.88	0.7444; 1.40	0.8444; 1.75	0.6600; 1.21	0.7711; 1.45

Each of these methods does not behave identically when comparing the nominal to realized FPR. At B = 0.0025 both the Macro and Tail methods run at about 1.03B, slightly over their budget. Some budgets exceed the nominal FPR when using Equal-FPR so the difference between nominal and realized FPR isn't just caused by the optimal allocation method. We plot these realized FPR values against the nominal in figure 1(a) and we can see how far off the diagonal line of equal rates they fall. Figure 1(b) shows the realized macro-recall for each method against their measured FPR.

If we set our nominal budget to B = 0.005. The macro recall for Equal-FPR is 0.8444. Both the Macro and Tail methods have a recall of 0.4644 and 0.4977. When looking at the realized federation FPRs they were 1.45B, 0.71B and 1.29B respectively.

We also looked at the variation of rate allocations when resampling the frontier benign and validation attacks scores per subtype. We took the unrestricted form of Macro and bootstrapped the scores 500 times for each seed and budget. We did not modify the train scorer or the test scores or the final calibration scores. For each client-seed-budget combination we found the percentage of rates that were different than the mode over the 500 bootstrap resamples. Averages over the 10 seeds and 9 clients give us 77.8%, 79.8%, 83.9%, 83.6% and 84.9% respectively as our budget increases. This number is averaged over 90 seed-client frequencies so it is not telling us the percentage of seeds that vary in rate. There is variation in the optimized decision given the amount of data on the frontier. Although these are not multiple instances of the system running nor does this describe how different they are from our original decision.

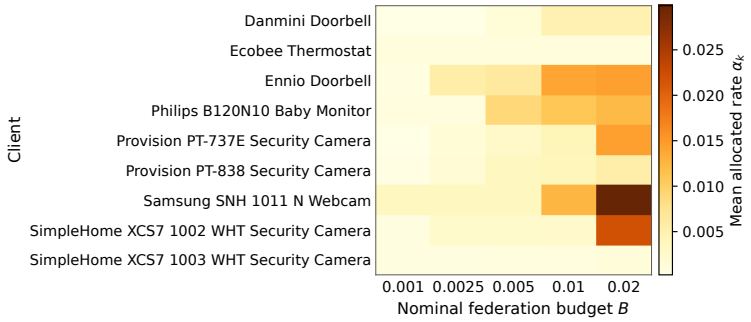


Fig. 2 Average rates chosen by the unrestricted Macro method for the N-BaIoT devices. This is averaged over 10 seeds of the original detector. Bootstrap resamples of the frontier are not shown.

Fig. 2: samsung webcam and simplehome xcs7 1002 security camera had the highest average rates when B = 0.020. Where the Danmini doorbell was given a lower rate.

5.2 Tail objective and worst-client behavior

Although Tail looks at the mean of the lowest 3 client utilities. All methods that fall under the FABRID family showed a minimum-client recall of zero. Table 1 shows this for all budgets. There were no attacks detected on the Danmini Doorbell device. For all unrestricted-policy operating points we tested. This is the cause of our minimum endpoint failing. There are also no alerts made on the Danminis benign held-out data and final calibration data. All 100 unrestricted-policy-seed-budget combinations gave an FPR of zero for the Danmini. The Danmini final calibration benign data has a length of 4,955 so at threshold $2/(4{,}955+1) = 0.000404$ we transition from making zero alerts.

We allocate rates that are both above and below this value. But we never find a rate that will detect attacks on this device when using unrestricted policy.

When looking at the ecobee device we see that at B = 0.020 the macro method allocates an average nominal rate of 0.001164. The ecobee gets a held-out benign FPR of 0.006672 and achieves 0.6000 macro attack recall. Despite the ecobee having a final calibration FPR of zero on the benign data.

We know that increasing our calibration sample size can increase the resolution of our rate threshold. But there could also be other factors causing the difference in alerts made on our test data. We don't know if these splits are i.i.d so we cannot say if there is a larger score distribution difference for some clients.

5.3 Primary contrasts and ranking ceiling

At a nominal budget of B = 0.001 the recall of Macro and Equal-FPR are the same. At B = 0.0025 there is a -0.0222 difference in recall. Which is not significantly different from zero. The recall for the Macro method is lower than Equal-FPR at larger budgets. Tail and Equal-FPR have a min client recall of zero until B = 0.005. When we reach B = 0.010 and B = 0.020 Equal-FPR can detect 0.9998 and 0.9999 min-client recall. But the Tail method still gets zero (See table 2). The realized FPR for Equal-FPR is also higher at these budgets.

Table 2 Seed level comparisons. Using 10 seeds we computed a 50,000 bootstrap CI and p-value using a sign flip test with holms correction over the 5 budget family.

	Macro − Equal-FPR: macro recall			Tail − Equal-FPR: minimum-client recall		
B	Mean Δ	95% CI	Holm p	Mean Δ	95% CI	Holm p
0.0010	0.0000	[0.0000, 0.0000]	1.0000	0.0000	[0.0000, 0.0000]	1.0000
0.0025	−0.0222	[−0.0556, 0.0111]	0.3594	0.0000	[0.0000, 0.0000]	1.0000
0.0050	−0.3800	[−0.4133, −0.3466]	0.0098	0.0000	[0.0000, 0.0000]	1.0000
0.0100	−0.4555	[−0.5000, −0.4111]	0.0098	−0.9998	[−0.9998, −0.9998]	0.0098
0.0200	−0.2556	[−0.2889, −0.2222]	0.0098	−0.9999	[−0.9999, −0.9999]	0.0098

Reallocation of thresholds cannot reorder scores so the AUROC and AUPRC metrics remain the same for all policy methods on the same seed. We calculate the AUROC and AUPRC after concatenating held out benign scores of all 9 devices with their eligible attacks test scores. We find the AUROC using rank and find the AUPRC by trapezoidal approximation. Our averaged results over the 10 seeds give an AUROC of 0.9995 and AUPRC of 0.9999. This metric calculates the score separation on the devices using all their test rows so devices are not weighted equally. Nor are all attack types weighted equally in the data pool. The total data pool contains 1,041,078 attacks and 111,188 benign samples giving us a prevalence of 90.35%. This will create higher base line performance on the precision-recall curve. We still have a min client recall of zero for all policy budgets that fall under the FABRID family when looking at these results.

Our CI and p-values are not controlling for a common FPR.

5.4 Secondary protocol stress test

For our secondary test we take the unrestricted form of macro and apply normal local calibration to the CIC IoT-DIAD dataset. Table 3 shows the average results over 10 scorers that were trained independently. There are 19 equally weighted clients that were trained using the reconstruction error as their anomaly score.

Table 3 CIC IoT-DIAD using unrestricted macro with local calibration. Scores are the reconstructed error and 19 equally weighted clients are shown over 10 seed averages.

B	Macro recall	Minimum-client recall	Realized FPR
0.0010	0.3614	0.0000	0.0098
0.0025	0.3970	0.0000	0.0118
0.0050	0.4318	0.0007	0.0160
0.0100	0.4602	0.0089	0.0213
0.0200	0.5016	0.0251	0.0311

As seen in our primary experiment we see our federation FPR go above the nominal value. At each increasing budget we have ratios of 9.80, 4.72, 3.20, 2.13 and 1.56. While our ratio decreases the measured FPR increases from 0.0098 to 0.0311. So our ratio

going down doesn't mean that our FPR is decreasing. Our secondary branch of this experiment does not include the Equal-FPR method. With the same scoring function and clients. We just want to see if we can also reproduce this nominal to realized FPR issue. This does not show if the Macro method outperforms the equal rate method at a comparable FPR.

6 Discussion

We did not evaluate whether reallocation of thresholds can outperform the equal rate baseline at the same measured FPR.

When B = 0.001 using a conservative method of calibration allows the measured federation FPR to stay under the nominal budget. But our attack recall decreases. There are no attacks detected on the Danmini doorbell for our macro and tail methods. So none of these allocations can be said to perform well. They all fall under our nominal FPR.

Because we used attacks to estimate our utility function. The FABRID-IDS process requires knowledge of attacks to apply our allocation to a problem. It cannot select client rates without labeled attack examples.

The amount of time to solve for an allocation was measured separately from our other processes. From 95.7ms to 358.8ms for Macro and 310.3ms to 3883.1ms for Tail. No solvers had to fallback on our allocation calculations. These times are not a true measure of our latency. We are not including our anomaly score time or our communication of information to other clients and computing alerts.

More research is needed to find if our algorithm can be deployed with malicious clients. Privacy concerns and communication of alerts and whether we can fix our score function to catch the attacks we missed.

Using B = 0.020 we see how claiming our recall outperforms another method can be misleading. Our Equal-FPR method is able to detect a macro recall of 1.0000 while running at a realized federation FPR of 1.88B. Whereas our Macro method detects 0.7444 while running at a FPR of 1.40B. So both methods are spending their nominal budget differently. We can only conclude that these two methods work at these levels. If they were both measured at the same FPR we don't know which would be better.

We have a low power test because of using only 10 seeds. Which gives us $2^{10} = 1024$ different signs we can assign. This leads to our smallest p-value being 0.0098 after using holm correction. With 5 tests in our family. The smallest possible p-value is 0.001953125 if we had all signs going in the same direction and no zeros. We saw this happen for the min client recall tests at budget 0.010 and 0.020. Because our Tail method had zero minimum-client recall and our equal-fpr had very little variation past 4 decimals. At B = 0.001 we have a zero-width CI and a p-value of one because our methods perform exactly the same. We can only correct for these p-values within each family and our CI is for each point.

Eq 3 can only give us a 95% statement about one clients FPR. If our samples are i.i.d then we can apply this to each client and have at most a 5% chance that the population-FPR bound fails for each client. We can apply a union bound to these nine clients and find we only have a lower bound of 1-9(.05)=.55 of hitting our thresholds

on all nine clients. Our data is not proven to meet these sample assumptions and we can't find a way to state our federation FPR has a 95% rate. So we don't know if our N-baiot dataset will allow for our rates to hit these bounds.

If we wanted to guarantee a certain false alarm rate we would need to test on benign samples that we think represent our monitor. Then we can see if our false alarms match our expectations. This is something that needs to be done if we want to monitor our rates as our system evolves.

The recall can change depending on your FPR rate. So it is important to keep that in mind when looking at the performance of our methods. Another factor is looking at the tail methods lower end of the client range. You can raise your performance on some of your worst clients but still have one client with no detections.

7 Conclusion

In this paper we proposed the FABRID-IDS process to allocate nominal FPR rates to our clients after training our detector. After our rates are picked we find the local threshold using our final calibration samples. The rates we measured after training did not always fall under the allocated federation budget. When B = 0.001 on the N-BaIoT dataset we saw the unrestricted versions of macro and tail had a measured federation FPR of .00170. When we lowered our rates with conservative calibration we were able to reach .00036. But lowered our attack recall. Our equal-fpr method detected more attacks at a few budget levels. But we were also higher on our measured FPR. We were not able to find any positive values for our min client recall for any of the FABRID family of methods. When we apply our process to the CIC IoT-DIAD dataset we find a similar trend of our nominal FPR differing from our measured. Though we don't have a comparison of the equal rate method. Our future work would like to see if we can apply our method while maintaining our false alarm rate in a monitored environment. Code and artifacts can be found at https://doi.org/10.5281/zenodo.22113172. Conflicts of Interest: The authors declare no conflict of interest.

References

[1] Nguyen, T.D., Marchal, S., Miettinen, M., Fereidooni, H., Asokan, N., Sadeghi, A.-R.: DÏoT: A federated self-learning anomaly detection system for IoT. In: IEEE ICDCS, pp. 756–767 (2019). 10.1109/ICDCS.2019.00080

[2] Wang, H., Muñoz-González, L., Eklund, D., Raza, S.: Non-IID data re-balancing at IoT edge with peer-to-peer federated learning for anomaly detection. In: ACM WiSec, pp. 153–163 (2021). 10.1145/3448300.3467827

[3] Hamdi, N.: Federated learning-based intrusion detection system for internet of things. International Journal of Information Security **22**(6), 1937–1948 (2023). 10.1007/s10207-023-00727-6

[4] Laridi, S., Palmer, G., Tam, K.-M.M.: Enhanced federated anomaly detection through autoencoders using summary statistics-based thresholding. Scientific Reports **14**(1), 26704 (2024). 10.1038/s41598-024-76961-2

[5] Asiri, M., Khemakhem, M.A., Alhebshi, R.M., Alsulami, B.S., Eassa, F.E.: Decentralized federated learning for IoT malware detection at the multi-access edge: A two-tier, privacy-preserving design. Future Internet **17**(10), 475 (2025). 10.3390/fi17100475

[6] Khan, M.A., Khalid, O., Rais, R.N.B.: Fed-DTCN: A federated disentangled learning framework for unsupervised zero-day anomaly detection in IoT with semantic-aware augmentation. Sensors **26**(6), 1918 (2026). 10.3390/s26061918

[7] Kairouz, P., McMahan, H.B., *et al.*: Advances and open problems in federated learning. Foundations and Trends in Machine Learning **14**(1-2), 1–210 (2021). 10.1561/2200000083

[8] Bridges, R.A., Jamieson, J.D., Reed, J.W.: Setting the threshold for high throughput detectors: A mathematical approach for ensembles of dynamic, heterogeneous, probabilistic anomaly detectors. In: IEEE BigData, pp. 1071–1078 (2017). 10.1109/BigData.2017.8258031
[9] Ochiai, H., Nishihata, R., Tomiyama, E., Sun, Y., Esaki, H.: Detection of global anomalies on distributed IoT edges with device-to-device communication. In: MobiHoc '23, pp. 388–393 (2023). 10.1145/3565287.3616528
[10] Tong, X., Feng, Y., Li, J.J.: Neyman–Pearson classification algorithms and NP receiver operating characteristics. Science Advances **4**(2), eaao1659 (2018). 10.1126/sciadv.aao1659
[11] Angelopoulos, A.N., Bates, S.: Conformal prediction: A gentle introduction. Foundations and Trends in Machine Learning **16**(4), 494–591 (2023). 10.1561/2200000101
[12] Lari, E., Arablouei, R., Werner, S.: Communication-efficient byzantine-robust federated conformal prediction via partial sharing. IEEE Transactions on Signal Processing, 1–16 (2026). Early access. 10.1109/tsp.2026.3737023
[13] Rockafellar, R.T., Uryasev, S.: Optimization of conditional value-at-risk. The Journal of Risk **2**(3), 21–41 (2000). 10.21314/jor.2000.038
[14] Meidan, Y., Bohadana, M., Mathov, Y., Mirsky, Y., Shabtai, A., Breitenbacher, D., Elovici, Y.: N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders. IEEE Pervasive Computing **17**(3), 12–22 (2018). 10.1109/MPRV.2018.03367731
[15] Rabbani, M., Gui, J., Nejati, F., Zhou, Z., Kaniyamattam, A., Mirani, M., Piya, G., Opushnyev, I., Lu, R., Ghorbani, A.A.: Device identification and anomaly detection in IoT environments. IEEE Internet of Things Journal **12**(10), 13625–13643 (2025). 10.1109/JIOT.2024.3522863